

BAHAN AJAR - PERTEMUAN 8

Strategi Algoritmik — Algoritma Greedy

TP	BK, AP — Strategi Algoritmik
----	------------------------------

A. APA ITU ALGORITMA GREEDY?

Greedy (rakus) adalah strategi algoritmik di mana kita mengambil **pilihan terbaik yang tersedia saat ini** (local optimum) dengan harapan akan menghasilkan **solusi terbaik secara keseluruhan** (global optimum).

Analogi Greedy

Situasi: Kalian di buffet — meja A ada sate, meja B ada nasi goreng, meja C ada es krim. Kalian lapar dan ingin kenyang maksimal.

Strategi	Cara
Greedy	Ambil makanan yang paling mengenyangkan <i>saat ini juga</i> , terus sampai kenyang
Non-Greedy	Pikirkan kombinasi makanan, hitung kalori, baru ambil

B. KARAKTERISTIK GREEDY

No	Karakteristik	Penjelasan
1	Greedy Choice Property	Pilihan optimal lokal → global optimal
2	Optimal Substructure	Masalah besar bisa dipecah jadi submasalah
3	Irrevocable	Sekali pilih, tidak bisa dibatalkan

Kapan Greedy Berhasil?

Matroid problems (graph, spanning tree)	Fractional knapsack
(boleh pecahan)	Penukaran koin dengan sistem mata uang standar
Activity selection (memilih jadwal terbanyak)	Huffman coding
(kompresi data)	Shortest path (Dijkstra)

Kapan Greedy Gagal?

× **0/1 Knapsack** (barang utuh, tidak boleh pecahan) × **Penukaran koin sistem ganjil** (misal pecahan 1, 3, 4 → tukar 6: Greedy ambil 4+1+1=3 koin, optimal 3+3=2 koin) × **Traveling Salesman Problem** × **Graph coloring**

C. PENUKARAN KOIN — CONTOH LENGKAP

Masalah

Tentukan jumlah lembar/koin minimal untuk menukar uang Rp11.800 dengan pecahan: Rp10.000, Rp5.000, Rp2.000, Rp1.000, Rp500, Rp200, Rp100

Pendekatan Greedy

Langkah	Sisa Uang	Pilih Pecahan	Alasan
1	Rp11.800	Rp10.000	Terbesar $\leq$ 11.800
2	Rp1.800	Rp1.000	Terbesar $\leq$ 1.800
3	Rp800	Rp500	Terbesar $\leq$ 800
4	Rp300	Rp200	Terbesar $\leq$ 300
5	Rp100	Rp100	Terbesar $\leq$ 100
	0		Selesai!

Hasil: $1 \times 10.000 + 1 \times 1.000 + 1 \times 500 + 1 \times 200 + 1 \times 100 = 5$ lembar

Algoritma dalam Pseudocode

```
PROCEDURE TukarKoin(M, koin[1..n])
  // M = jumlah uang, koin = pecahan terurut menurun
  sisa  $\leftarrow$  M
  hasil  $\leftarrow$  []
  FOR i  $\leftarrow$  1 TO n DO
    WHILE sisa  $\geq$  koin[i] DO
      hasil  $\leftarrow$  hasil + [koin[i]]
      sisa  $\leftarrow$  sisa - koin[i]
    ENDWHILE
  ENDFOR
  RETURN hasil
END
```

Contoh Lain

Jumlah Uang	Hasil Greedy	Jumlah Koin
Rp27.500	$20.000 + 5.000 + 2.000 + 500$	4
Rp33.700	$20.000 + 10.000 + 2.000 + 1.000 + 500 + 200$	6
Rp45.900	$20.000 + 20.000 + 5.000 + 500 + 200 + 200$	6

D. FRACTIONAL KNAPSACK

Masalah

Tas kapasitas 50 kg. Tersedia barang dengan harga per kg berbeda. Kita boleh mengambil **pecahan** barang.

Barang	Berat (kg)	Harga/kg	Total Harga
Emas	30	Rp60.000.000	Rp1.800.000.000
Berlian	5	Rp100.000.000	Rp500.000.000
Mutiara	10	Rp25.000.000	Rp250.000.000
Perak	20	Rp10.000.000	Rp200.000.000

Langkah Greedy — Urutkan Density (Harga/kg) Tertinggi

Rank	Barang	Density	Berat	Ambil	Total Ambil
1	Berlian	Rp100jt/kg	5 kg	5 kg (ambil semua)	5 kg
2	Emas	Rp60jt/kg	30 kg	30 kg (ambil semua)	35 kg
3	Mutiara	Rp25jt/kg	10 kg	10 kg (ambil semua)	45 kg
4	Perak	Rp10jt/kg	20 kg	5 kg (sisa kapasitas)	50 kg

Hasil Optimal: - Berlian: 5 kg × Rp100jt = Rp500jt - Emas: 30 kg × Rp60jt = Rp1.800jt - Mutiara: 10 kg × Rp25jt = Rp250jt - Perak: 5 kg × Rp10jt = Rp50jt - **Total: Rp2.600.000.000**

Algoritma Pseudocode

```

PROCEDURE FractionalKnapsack(W, barang[1..n])
    // W = kapasitas tas
    Urutkan barang berdasarkan density (harga/berat) menurun
    totalNilai ← 0
    sisaKapasitas ← W

    FOR i ← 1 TO n DO
        IF sisaKapasitas ≥ barang[i].berat THEN
            totalNilai ← totalNilai + barang[i].totalHarga
            sisaKapasitas ← sisaKapasitas - barang[i].berat
        ELSE
            fraction ← sisaKapasitas / barang[i].berat
            totalNilai ← totalNilai + fraction × barang[i].totalHarga
            BREAK
        ENDIF
    ENDFOR

    RETURN totalNilai
END

```

E. JADWAL KEGIATAN (ACTIVITY SELECTION)

Masalah

Ada 5 kegiatan dengan waktu mulai dan selesai. Pilih kegiatan sebanyak-banyaknya tanpa tumpang tindih!

Kegiatan	Mulai	Selesai
A	08:00	10:00

Kegiatan	Mulai	Selesai
B	09:00	11:00
C	10:30	12:00
D	11:30	13:00
E	13:00	14:00

Strategi Greedy: Pilih kegiatan yang selesai paling awal!

1. **A** (08:00–10:00) — selesai 10:00
2. **C** (10:30–12:00) — mulai $\geq$ 10:00
3. **E** (13:00–14:00) — mulai $\geq$ 12:00

Hasil: A, C, E — 3 kegiatan (optimal)

Algoritma Pseudocode

```

PROCEDURE ActivitySelection(jadwal[1..n])
    Urutkan jadwal berdasarkan waktu selesai menaik
    pilih ← [jadwal[1]]
    lastSelesai ← jadwal[1].selesai

    FOR i ← 2 TO n DO
        IF jadwal[i].mulai ≥ lastSelesai THEN
            pilih ← pilih + [jadwal[i]]
            lastSelesai ← jadwal[i].selesai
        ENDIF
    ENDFOR

    RETURN pilih
END

```

F. KELEBIHAN & KELEMAHAN GREEDY

Kelebihan	Kelemahan
Sederhana — mudah dipahami	× Tidak selalu menghasilkan solusi optimal
Cepat — $O(n)$ atau $O(n \log n)$	× Hanya cocok untuk masalah tertentu
Cocok untuk masalah besar	× Sulit dibuktikan optimalitasnya
Iteratif — tidak perlu rekursi	× Keputusan awal bisa menyesatkan

G. RANGKUMAN

Konsep	Inti
Greedy	Ambil pilihan terbaik saat ini (local optimum)
Penukaran Koin	Ambil pecahan terbesar yang $\leq$ sisa
Fractional Knapsack	Ambil barang density tertinggi dulu

Konsep	Inti
Activity Selection	Pilih kegiatan yang selesai paling awal
Gagal	Jika pilihan lokal → bukan global optimum

MGMP Informatika SMAN 6 Cimahi